

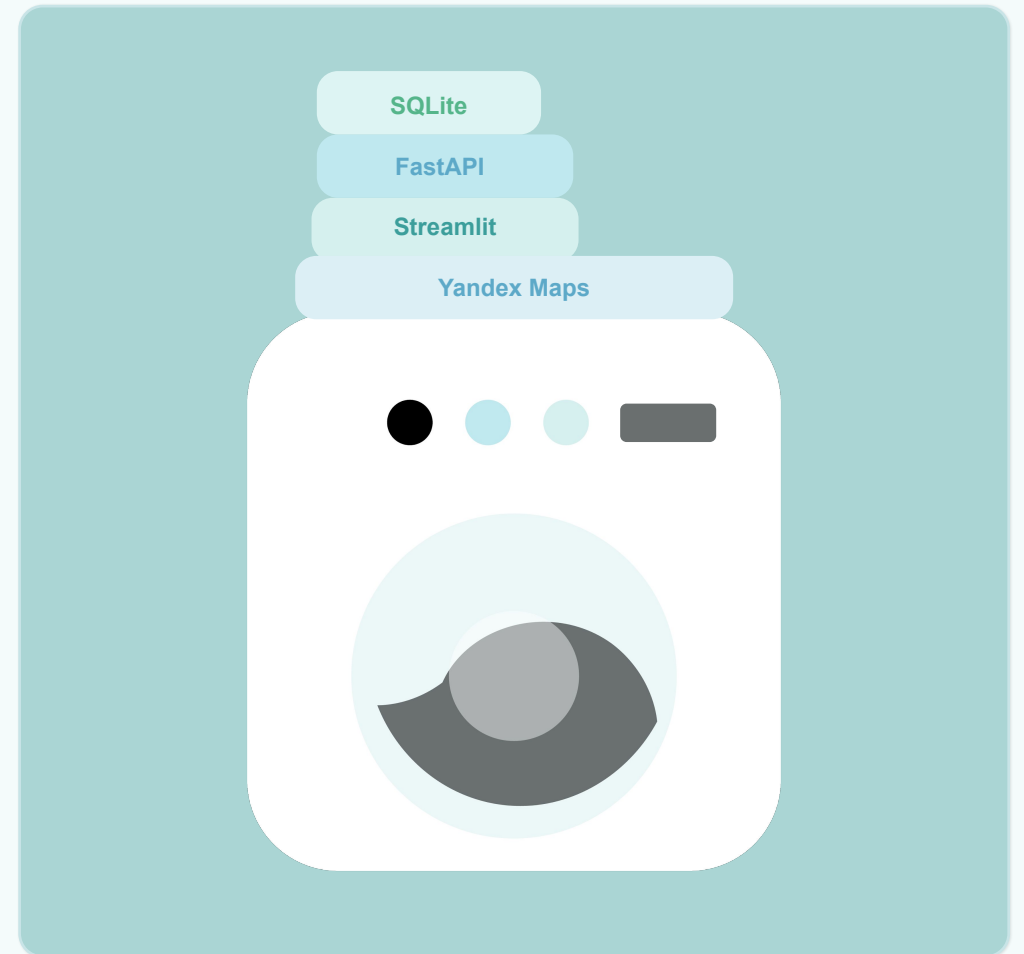
Project Presentation: 13.04.2026

Safe & Sound laundry monitor

Project by Veronika Levasheva, Dmitriy Lukianov, Makar Dyachenko, Olesya Novoselova and Ulyana Yanovitskaya

Laundry monitoring system with automated quality gates

Software Quality, Reliability and Security course (Innopolis University BS23) by Dr. Andrey Sadovykh



What's on for today?

1

1. Quality Gate Automation

Live CI pipeline, checks, PR blockers

2

2. Methods Applied

Static, dynamic, security and performance techniques

3

3. Results & Lessons Learned

What worked, what still needs improvement

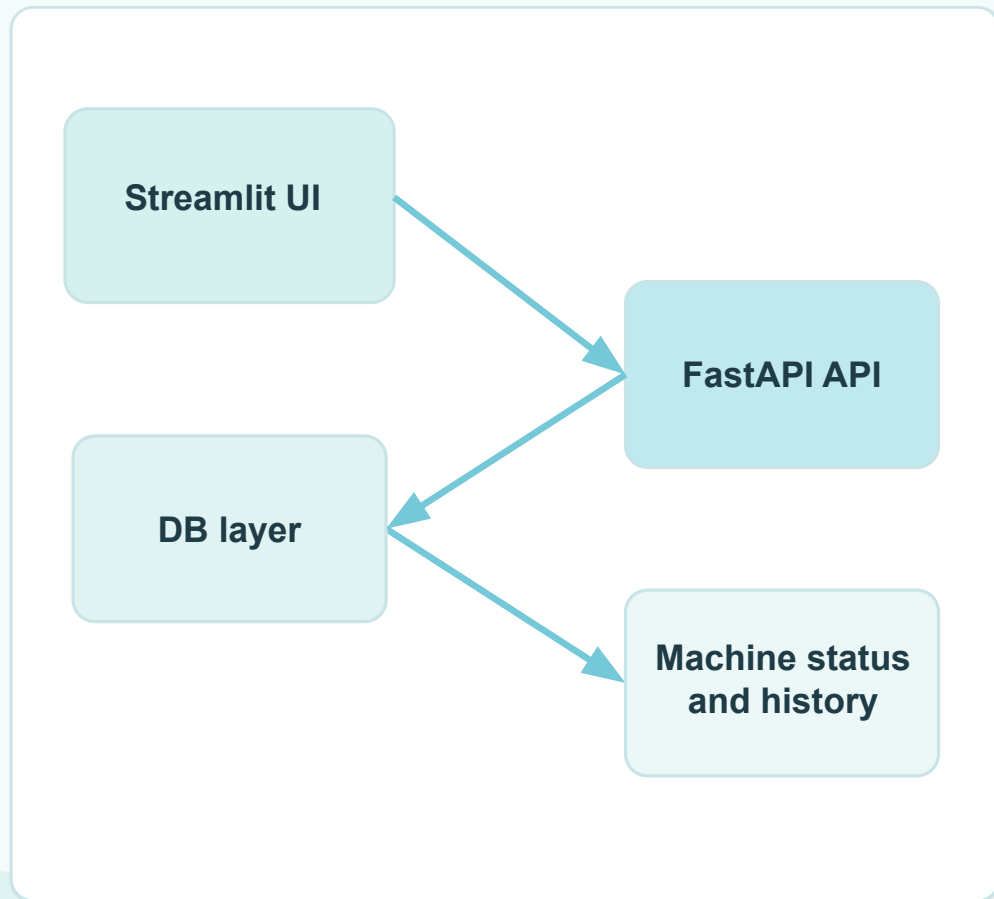
4

4. Teamwork

Who did what and how the team split the work

Project at a glance

Small product, clear architecture, quality-first workflow



Core product scope

- Dorm washer/dryer availability tracking
- Report submission with remaining time
- History endpoint for raw reports
- Optional Yandex Maps widget integration

Main API endpoints

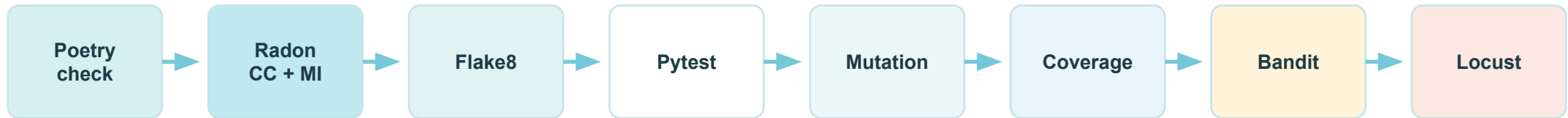
GET /machines

POST /report

GET /history

1. Quality Gate Automation

A pull request is checked automatically before merge



What the gate gives us

- Fast feedback on every push and PR
- Same checks for all contributors
- Visible blockers before merging to main

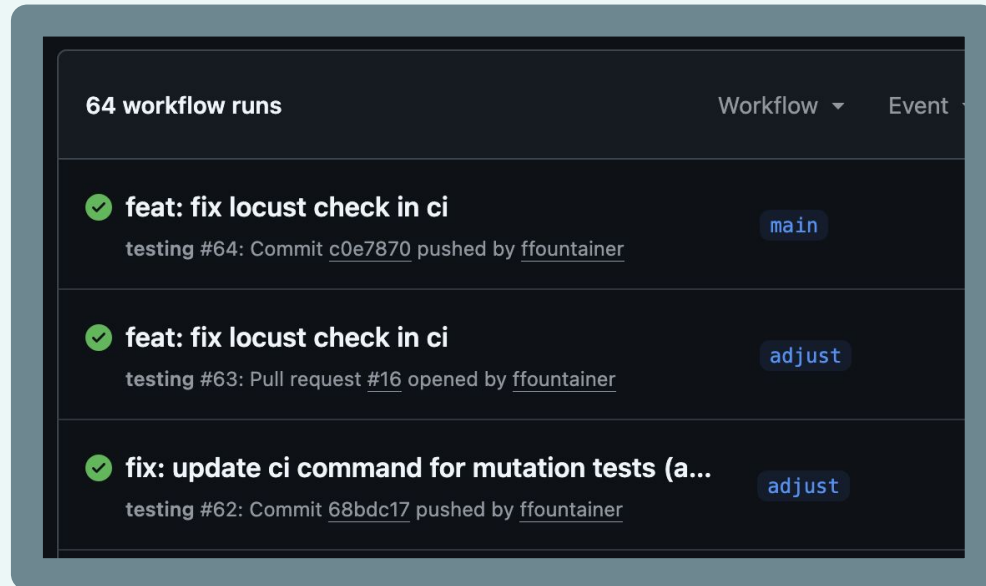
Merge blockers we observed

- Maintainability index can be improved (right now we have one B)
- Initially our locust CI setup was invalid, and we couldn't see the valid state of our app performance
- Flake often gave PEP8 errors since we were forgetting to lint

Live pipeline demo!

For demo

- Open GitHub Actions and show the quality gate
- Show one branch → PR workflow example
- Open Swagger /docs and key endpoints
- Show current frontend/backend integration state



2. Methods Applied

Breadth of techniques across code quality, testing, security and performance

Static analysis

- PEP8 style check
- Cyclomatic complexity
- Maintainability index
- Dependency validation

Dynamic testing

- Unit tests
- Computational tests
- Property-based tests
- Mutation testing

Security

- Bandit scan

Performance

- Locust smoke-test path
- CI integration

3. Results & Lessons Learned

Solid progress, but not all quality gates are stable yet

Completed areas

CI/CD, API, DB layer, mutation + property-based testing, unit tests, security, etc (everything that was mentioned in the file)

In progress

Style and cyclomatic complexity need constant review during updates

Needs follow-up

Aim for higher maintainability index, check other parts of the testing suite with mutations; implement CD workflow

What worked well

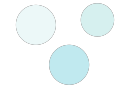
- Small project scope made integration manageable
- Tasks were divided into two weeks: first was for implementation, second for testing
- Automation helped to expose issues early
- Everyone worked in their own branches, and main stayed organized

What we would improve next

- Start monitoring test coverage earlier (we checked it in the pipeline right before the deadline)
- Keep google sheet updated after each completed task
- Add CD pipeline for checking if the application runs correctly

4. Teamwork

Work was distributed by system layer and by QA technique



Member

Main contribution areas

Veronika Levasheva
@fountainr

CI pipeline, mutation testing, style/complexity checks, team management

Makar Dyachenko
@index099

DB layer, unit tests, maintainability, performance / Locust work

Olesya Novoselova
@doiwannaknoww8

API, external maps API, Hypothesis tests, OpenAPI, security scan with Bandit

Dmitriy Lukianov
@hntlvz

Frontend, E2E tests, domain error tests, dependency validation tasks

Ulyana Yanovitskaya
@polsha_byaka

Computational tests, README.md, presentation creation

Everyone did well !

Next step (and maybe app demo?)

Next priorities and improvements

- Make maps integration more meaningful
- Stabilize coverage and smoke-test setup
- Keep PR workflow and documentation consistent

And if we have enough time, we will show you how our app really looks like!

And if not? Check it out in our [repo](#) yourself :))

Thank you <3